UNIVERSITY of CALIFORNIA
RIVERSIDE

A Web Application for Database Exploration by Relative Image Similarity (DEBRIS)

A Thesis submitted in partial satisfaction of the
requirements for the degree of

Master of Science

in

Engineering

by

Kevin Sanford

September 2025

Thesis Committee:

Dr. Bahram Mobasher, Chairperson
Dr. Saeed Rezaee

The Thesis of Kevin Sanford is approved:

Committee Chairperson

University of California, Riverside

Acknowledgements

I would like to thank my wife for always believing in me and supporting my efforts, even when I choose difficult paths. I am extremely fortunate to be able to share this life with you.

Thank you Dr. Bahram Mobasher and Dr. Saeed Rezaee for your support with developing my thesis. With your guidance and encouragement, I have been able to overcome obstacles that I was not sure I could overcome.

I would also like to thank my friend Krish Chowdhary for his advice regarding software development, career development, and life in general. Your encouragement and humor have helped motivate me to continue pursuing my goals, even when it seems impossible at times.

ABSTRACT OF THE THESIS

A Web Application for Database Exploration by Relative Image Similarity (DEBRIS)

by

Kevin Sanford

Master of Science, Graduate Program in Engineering
University of California, Riverside, September 2025
Dr. Bahram Mobasher, Chairperson

Image retrieval techniques typically involve a text-based search, though image-based search may also be employed. Image retrieval from an image search requires the use of computer vision algorithms to analyze the image query and relate it to images in a database. The design of these algorithms depends on the specific goals of the search engine. Image searches that aim to retrieve near-exact matches generally use computationally expensive algorithms, while searches focused on visual similarity over exactness may use simpler algorithms. The algorithm design also depends on the database to be searched. Very large databases, such as those built from the public domain, require storage and maintenance in data centers, which incurs additional computational costs. Algorithms for these public domain databases must be optimized for speed. A search through a custom database provides a more streamlined search through information that may not necessarily be public. When using these algorithms for browsing a database, retrieving visually similar images becomes a key objective.

The Database Exploration By Relative Image Search (DEBRIS) web application is designed to search a large collection of images to retrieve those similar to a query image submitted by the user. The application begins by prompting the user to upload a folder of images to populate a custom database. The uploads are first processed through a computer vision feature extraction pipeline, which generates vector embeddings for each image. Each embedding represents the image's location in a 768-dimensional vector space. After the feature extraction process, the images are organized using a self-organizing map (SOM) - an unsupervised machine learning algorithm that reduces the dimensionality of the feature space to two dimensions, while preserving its topography. The map consists of nodes that span the newly defined two-dimensional space, and the images in the database are mapped to the space and assigned to the closest node. As a result, each node represents a cluster of images in close proximity in the multidimensional space.

Once the map is created, the user is prompted to upload a query image with which they intend to retrieve similar images from the newly created and organized database. The query image is processed using the same feature extraction algorithm and mapped to the two-dimensional space defined by the database. The mapping process involves identifying a node that corresponds to the location of the query image. That node is then used to retrieve images that are similarly categorized. The embeddings of the retrieved images are then compared to the embeddings of the query image using cosine similarity. The algorithm ranks each image in the node to display those with the highest cosine similarity score. Unlike algorithms such as the K-nearest neighbors algorithm, this algorithm only calculates a measure of similarity between data points that are grouped together by the SOM. This

approach enables faster retrieval, since the data points are pre-organized, eliminating the need to compute the distances between the query image and every data point for every search.

To allow the user to explore the database, a few of the images retrieved are randomly selected from neighboring nodes. These images are loosely related to the others, since the SOM arranges nodes such that their distance in the two-dimensional space correlates with their distance in the original high-dimensional feature space. The user can then continue exploring the database by selecting one of the retrieved images to view other images similar to it. Since the retrieved images are ranked by similarity, the user may either browse within the space of similar images by selecting from the first few images displayed, or explore further from the starting node by selecting from the last images displayed. This provides the user with efficient and intuitive control over their navigation through the database.

Grouping similar images based on the nodes of the self-organizing map, rather than the actual distances between images in their high-dimensional space, allows the algorithm to emphasize visual similarity over exactness. If an image retrieval system that uses an SOM requires a high degree of accuracy, it would need to evaluate the distances between a significant portion of the data points for a more comprehensive search of the image with the closest match to the query, since the most accurate match might be categorized into a neighboring node. Because DEBRIS was not intended to be optimized for accuracy, the methods required for a higher degree of accuracy may be traded for less computationally expensive requirements, facilitating the development of a relatively fast, lightweight application such as DEBRIS. A simpler, computationally efficient algorithm, such as the self-organizing

map, is well-suited for the architecture of an image search application focused on database exploration rather than the precise identification of specific data points.

DEBRIS is an open source web application that gives users the freedom to choose their own collection of images to browse, based on the visual similarity within the set. It is designed to provide the user with the ability to browse a database created from their images in an intuitive and user-friendly manner without the need for labels or metadata. By providing the ability to browse a collection of images by visual elements alone, DEBRIS would be particularly useful for finding visual motifs in art galleries, fashion catalogs, and other visual-based media.

Contents

List of Figures

List of Abbreviations

CAE	Convolutional autoencoder
CBIR	Content-based image retrieval
CNN	Convolutional neural network
DEBRIS	Database Exploration by Relative Image Similarity
KNN	K-nearest neighbors
ML	Machine learning
SOM	Self-organizing map

1

Introduction

1.1 Overview

There have been many approaches to the design of image retrieval systems, even before the advent of computing machines. For example, a photo album may be organized to facilitate efficient retrieval of personal photographs. Traditional image retrieval systems such as the Library of Congress Thesaurus for Graphic Materials and the Netherlands Institute for Art History's Iconclass provided algorithms before the age of digital computing to organize images by their features and context to enable a convenient means to find them (Rafferty, 2019). These early examples involve systems for examining images, organizing them by their content, and providing a means of retrieval. With modern advances in computing technology, image retrieval systems have since been developed for more efficient execution.

In early digital image retrieval systems, users were required to submit a text query in order for the system to return images associated with related labels and metadata (Gail-

lard et al., 2018). With the popularization of social media, an uncountable number of images have been uploaded to the internet, creating a need for content-based image retrieval (CBIR) systems that extract features from images as an alternative labeling scheme (Gaillard et al., 2018; Rafferty, 2019). Since CBIR systems extract features from images without the need for human action to label them, the search query itself could also be an image, where the search terms are the features extracted from the image. Early CBIR systems were limited to returning near-exact matches of the query image, but more advanced computer vision algorithms, particularly those built with deep neural networks, have been used to successfully retrieve images with emphasis on similarity rather than exactness (Kiran et al., 2022; Singh et al., 2023).

A major focus of CBIR research is the efficient storage and retrieval of information using suitable databases (Kiran et al., 2022). The design of a CBIR system is heavily dependent on its function. For example, a system designed to detect specific features in medical images must preserve certain detailed information, so a computationally expensive algorithm such as K-nearest neighbors (KNN) may be used to minimize information loss (Sampathila et al., 2022). For more general-purpose systems, a focus on dimensionality reduction of the vectors in the image feature space provides an effective means to improve efficiency (Singh et al., 2023). Additionally, a CBIR system built with efficiency in mind should utilize efficient data management solutions, such as SQL-based database engines (Gaffney et al., 2022).

The Database Exploration by Relative Image Similarity (DEBRIS) web application is a CBIR system that was designed to allow users to efficiently browse a custom database

of images. The user is prompted to upload a large collection of images, which is then stored in an SQLite database along with the features extracted from a deep neural network with the use of transfer learning. A self-organizing map (SOM) is then applied to the database to further reduce the dimensionality of the vectors representing each image, while preserving the topology of the image feature space. The two-dimensional projection of the image feature space created by the SOM provides a means of efficient retrieval of related images by defining clusters in the space and assigning each image to a corresponding cluster.

To browse a database with DEBRIS, the user begins by uploading a single image as a search query. The image query then undergoes the same feature extraction process as the images in the database. The features are then mapped to the SOM to efficiently retrieve similar database images from the SOM clusters. After a limited number of images are retrieved, the high-dimensional features of the images are compared to rank and display the results by relative similarity. The user can then select one of the 10 images displayed to retrieve an additional 10 images similar to the selected image. Through this iterative process, a user may visually browse the database by relative image similarity.

1.2 Significance

Popular CBIR systems, such as Google Images and TinEye, are geared toward image searches in the public domain and often rely on labeled images or semantic grouping to supplement the content-based search. Services that allow users to search a collection of their own images are not open source and usually require a subscription. The iPhone Photos application enables users to search through personal photos using keywords, date/location

metadata, or facial recognition, but it currently lacks a mechanism for browsing by visual similarity to an image selected by the user.

DEBRIS was developed as an open source web application that gives users the freedom to choose their own collection of images to browse, based on the visual similarity within the set. Since the application is designed to operate entirely without the use of text-based features, no labels or metadata are required. The user simply uploads a collection of images to create a database, then uploads a single image to retrieve similar images from the database. They could continue visually browsing the database by selecting one of the retrieved images to repeat the process. DEBRIS was created to facilitate a more intuitive image browsing experience by allowing users to search using visual elements, rather than relying on explicit search terms. This feature is particularly useful for finding visual motifs in image collections that are more difficult to describe semantically, such as those found in art galleries and fashion catalogs.

2

Literature

2.1 Previous Research

Digital image retrieval systems are generally divided into two categories: text-based image retrieval and content-based image retrieval. This division, commonly known as the "semantic gap," is the subject of much research related to CBIR because it poses the challenge of bridging the human perception of images with the way computers process them (Hai et al., 2023; Hameed et al., 2021). Research interest in bridging this gap is derived from the requirement of most image retrieval systems to provide an intuitive interface for a user to describe their search terms. Heesch (2008) argues that a CBIR system with a more intuitive and interactive interface may offer a superior alternative to text-based systems altogether. However, the most effective design for an image retrieval system ultimately depends on its function.

Many image retrieval systems have been designed to serve specific needs across a variety of fields. For example, Seo et al. (2023) developed an image retrieval system

with the express purpose of classifying galaxies based on morphological type. To meet the requirements of this system, a convolutional autoencoder (CAE) was trained to extract the morphological features of galaxies from images (Seo et al., 2023). Other systems may focus on different design features to suit their purpose. In general, engineers should consider these key design aspects of a CBIR system to ensure that it fulfills its purpose: the data used, the features extracted from the data, the method by which features are organized into a database, and the user interface/visualization method.

The design of an image retrieval system determines the relevance and importance of the type of data used. Since the system created by Seo et al. (2023) was designed specifically to analyze images of galaxies, it may not produce meaningful insight when applied to unrelated data. Systems aimed at bridging the semantic gap in the field of image retrieval often utilize datasets of labeled images (Laaksonen et al., 2004; Sampathila et al., 2022; Singh et al., 2023). One notable example is the image retrieval system created by Hai et al. (2023), which uses an unsupervised machine learning algorithm to extract high-level semantics from images that are then used to organize the data.

The method of feature extraction is a defining element of an image retrieval system, since its structure depends on the extracted information. Classic CBIR algorithms extract unique features of images, such as color, texture, and shape, by several methods that include the calculation of color moments, the definition of statistical textures, and edge detection (Gaata & Hantoosh, 2018; Laaksonen et al., 2004; Lin et al., 2008; Meng et al., 2016; Wang et al., 2015). The popularization of deep neural networks has accelerated CBIR research because these models have proven to extract image features more accurately than classic

methods (Hameed et al., 2021). Several flavors of deep neural networks have been applied to systems related to astronomy, online shopping, and CBIR research (Hai et al., 2023; Katasani et al., 2019; Kiran et al., 2022; Seo et al., 2023; Singh et al., 2023). The concept of transfer learning has been especially useful for image feature extraction using deep neural networks because it eliminates the need to train a full neural network model by using the weights defined by a pre-trained model. This approach greatly reduces the computational expense of characterizing an image dataset, which was a goal of Katasani et al. (2019), Kiran et al. (2022), and Singh et al. (2023) in the design of their image retrieval systems. The pre-trained convolutional neural network (CNN) ResNet is a common example of the popularity of transfer learning because it demonstrated breakthrough accuracy in computer vision tasks, providing improved performance for systems that use the model (He et al., 2015). The development of vision transformers introduced yet another type of deep neural network that performs better than CNNs when trained on large datasets (Dosovitskiy et al., 2020).

Once features are extracted from a collection of images, a database storage scheme must be selected to meet the requirements of the image retrieval system. An intuitive first approach to database storage is the creation of an index of the extracted features for each image. For a database populated in this manner, a common retrieval method involves the use of a KNN algorithm to calculate the similarity between the image query and every other image in the database using similarity measures such as Minkowski distance, Euclidean distance, and cosine similarity (Kiran et al., 2022; Sampathila et al., 2022; Seo et al., 2023). This method is computationally expensive, especially for a large database,

since it requires comparing the query to every data point. The computational cost may be reduced by further processing the data extracted from the images, which usually involves dimensionality reduction and clustering techniques, such as SOMs and k-means clustering (Katasani et al., 2019; Laaksonen et al., 2004; Lin et al., 2008; Meng et al., 2016; Singh et al., 2023). These techniques reduce the number of data points for measuring similarity scores between images, therefore reducing the time complexity of the image retrieval system overall.

The final consideration in the design of an image retrieval system is the user interface. For a CBIR system that retrieves images to classify the image query, a display of the retrieved images would not be necessary, as seen in the search engines developed by Sampathila et al. (2022) and Seo et al. (2023) to classify medical images and images of galaxies, respectively. However, for a system designed to provide the user with the ability to browse a database, the retrieved images should be displayed in an intuitive and user-friendly manner. The retrieved images are usually limited in number and displayed in order of similarity ranking (Katasani et al., 2019; Kiran et al., 2022; Laaksonen et al., 1999; Singh et al., 2023). In general, a display of a few images for each query would be sufficient to engage the user in database browsing; however, more sophisticated interfaces have been developed, such as the iMap tool, developed by Wang et al. (2015) to visualize similar images in a mosaic-like fashion.

2.2 Findings and Unanswered Questions

Research into CBIR systems is often geared toward a specific purpose, such as image classification for medical or astronomical research purposes (Sampathila et al., 2022; Seo et al., 2023). More general CBIR systems do not typically provide an open source repository for public use (Katasani et al., 2019; Kiran et al., 2022; Lin et al., 2008; Meng et al., 2016; Singh et al., 2023; Wang et al., 2015). PicSOM, the CBIR system developed by Laaksonen et al. (1999), was once published online for public use, but it has since been taken offline. As a result, none of these CBIR systems are available to browse a user-designated collection of images. Additionally, since much of the CBIR research is focused on closing the semantic gap, not as many systems exist to explore a database built from unlabeled collections of images.

DEBRIS was developed as a general-purpose image database exploration tool that can be applied to a collection of images supplied by the user without the need for prior labeling. Since this application runs on the user's machine, limits on the size of the database and the processing power necessary to create it are governed by the limitations of the machine. The system limits file types to only accept JPEG, PNG, BMP, and TIFF files. The uploaded images may take any shape, since they will be resized in the pre-processing phase, which limits retention of certain details in large and non-square images. The ability to supply a dataset of unlabeled images of any size provides a degree of customization that affords the user flexibility and creativity while using the product. This approach to a CBIR system does not appear to exist elsewhere in the literature.

3

Methodology

3.1 Preliminary Work

The development of the DEBRIS web application began with an exploration of the SOM algorithm, which uses an artificial neural network to represent a two-dimensional projection of the topography of data points in a high-dimensional space (Kohonen, 1982). This dimensionality reduction provides a means of intuitive visualization, creating a map that places data points with similar attributes together, while data points with dissimilar attributes are placed further apart. The map is composed of nodes that are fit to these data clusters, giving the map its structure. To illustrate how an SOM organizes information, I used the sklearn-som Python package developed by Smith (2021) and applied it to the Fashion-MNIST dataset of labeled, pre-processed clothing item images created by Xiao et al. (2017). Figure 3.1 shows a sample of the dataset, but since SOMs are unsupervised machine learning algorithms, the labels are not important to organize the information.

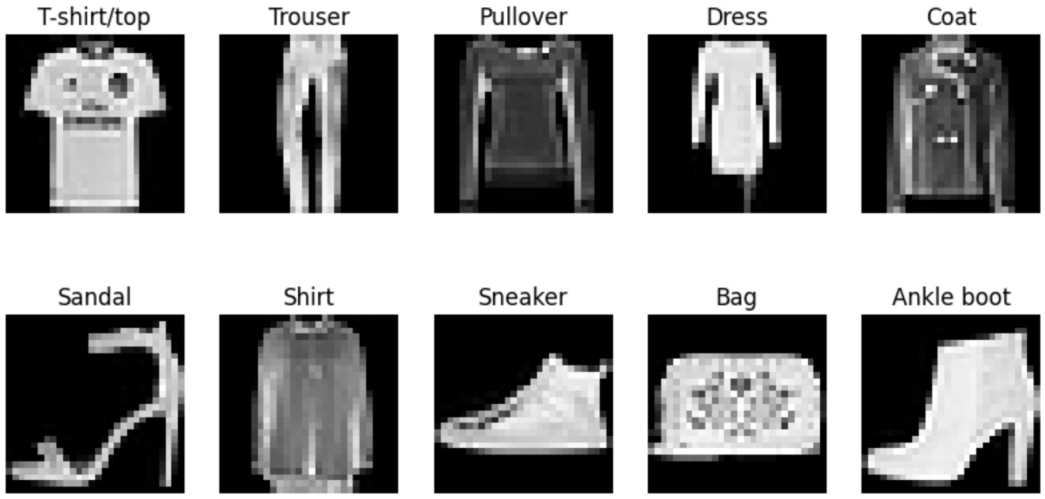

Figure 3.1: Example images from each category labeled in the Fashion-MNIST dataset created by Xiao et al. (2017).

To organize data with the SOM algorithm, each data point processed through the algorithm is required to be defined by a vector of the same length as all other data points in the dataset. The images in the Fashion-MNIST dataset are pre-processed by taking real photographs of clothing items, cropping and centering them, resizing them into 28-by-28-pixel squares, emphasizing edges with a Gaussian operator, then converting the pixel values to 8-bit grayscale values (Xiao et al., 2017). The result of this pre-processing method is a collection of images, each represented as a flattened array of 784 integer values. My first approach to exploring the SOM algorithm was to use these integer values as a vector that defines the location of each of these images in a 784-dimensional space. The result of applying the SOM algorithm to the Fashion-MNIST dataset is visualized in Figure 3.2, where I initialized a 10-by-10 grid of nodes and ran the algorithm to fit the data. After the weights of the nodes were established, a prediction was applied to each image to determine which node it lies closest to on the map. To better understand the content assigned to

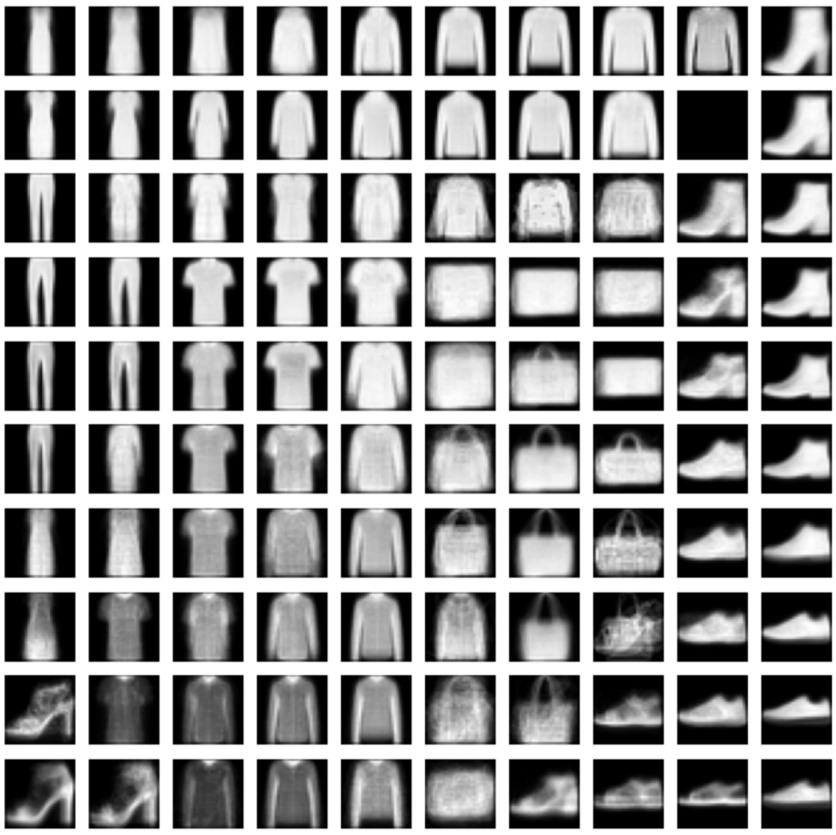

Figure 3.2: Map of composite images for each node in a self-organizing map. Each composite image is comprised of all of the images assigned to the node. The empty node displayed is absent of information. The composite images are derived from the Fashion-MNIST dataset created by Xiao et al. (2017).

each node, I took the average pixel values for each pixel location across every image at a node, which produced the composite images in Figure 3.2. The figure shows that images with similar shapes were clustered together in the map, such as the footwear on the right-hand side. For a more comprehensive analysis of the map's arrangement, hyperparameter optimization may be performed to produce heatmaps of the image types.

Since the SOM could categorize these pre-processed images into a two-dimensional space effectively, it seemed that it could be used for an efficient CBIR system because images

could be retrieved from a limited region on the map without the need to evaluate each data point. In the Fashion-MNIST example, the pre-processed images emphasize shape, so the SOM organized the map with an emphasis on shape, which implies that the features intended to be highlighted in the pre-processing phase govern the organization method of the SOM. If a user of a CBIR system wanted to retrieve images of a certain clothing shape, this map could be a useful part of the system, where a node is selected and images assigned to the node are returned. Then the desired shape would be the query in the CBIR system, and the images returned from the node would be the system response. To submit a query, the user would either supply an image or select one from a list. The image query would need to have undergone a similar pre-processing method as the database in order to be organized into the map accurately. This requirement is easily satisfied when supplying the user with a list of images from the Fashion-MNIST dataset, but to provide the user with the freedom to upload their own image as a query, it would first need to be pre-processed.

To develop a preliminary CBIR system using an SOM and the Fashion-MNIST dataset, I used Flask, which is an open source web application framework that is ideal for simple applications (Pallets, 2025). Flask provides a means to build a user interface that allows the user to upload an image and view the results of their search. To pre-process an uploaded image, the CBIR system I built applies a Gaussian operator, crops, resizes, and converts the image to 8-bit grayscale in a process similar to that used by Xiao et al. (2017). After pre-processing, the program maps the image to the saved model created by the SOM to find its closest node and retrieves seven random images from that node. To give the user the ability to explore the map outside of a single node, it also retrieves three random images

from adjacent nodes. Each of the retrieved images may be selected by the user as the next image query to repeat the process. Figure 3.3 displays the result of an initial image upload, and Figure 3.4 shows the result of selecting the next image query from the list of retrieved images. Selecting one of the first seven retrieved images would display similar results from the same node, while a selection of one of the last three retrieved images would display results from an adjacent node.

The concepts explored in this preliminary CBIR system form the foundation for the DEBRIS web application. To expand this system, I decided to incorporate three key elements: the use of a deep neural network for more detailed feature extraction, proper database handling for efficient storage and retrieval, and the ability for a user to upload their own collection of images to populate the database. With these features, a CBIR system could be designed to allow a user to visually browse their own image dataset without prior labeling. To ensure the practicality of a system intended for public use, its design should emphasize computational efficiency.

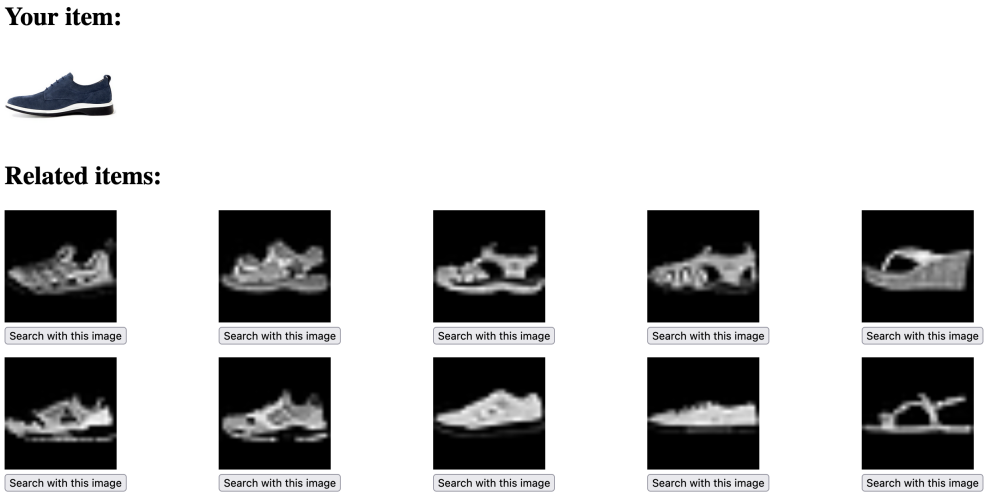

Figure 3.3: Result of the preliminary content-based image retrieval system built from a self-organizing map applied to the Fashion-MNIST dataset. The image query was uploaded by the user. Seven images were retrieved from the node assigned to the image query, and three images were retrieved from adjacent nodes.

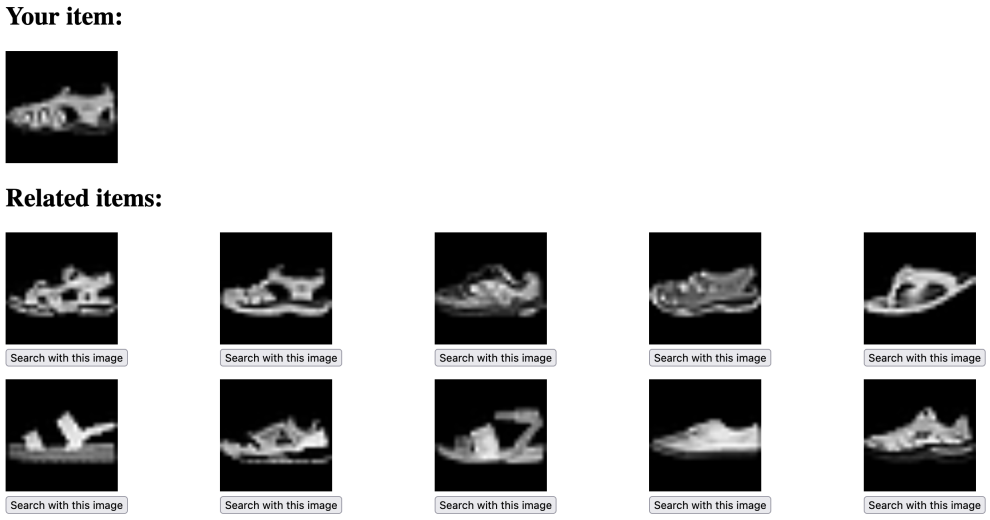

Figure 3.4: Result of the preliminary content-based image retrieval system built from a self-organizing map applied to the Fashion-MNIST dataset. The image query was selected by the user from the results in Figure 3.3. Seven images were retrieved from the node assigned to the image query, and three images were retrieved from adjacent nodes.

3.2 Approach

DEBRIS was designed as a CBIR system that allows a user to visually browse their own image dataset without prior labeling while maintaining a practical computational expense. To begin building DEBRIS from my preliminary work, I rewrote the initial CBIR system in Django because it is an open source web application framework like Flask, except that it is geared toward creating and handling custom databases (Django Software Foundation, 2025). Django allows users to choose which database management systems to utilize, so I chose to use SQLite to meet DEBRIS's operational requirement of efficient database creation, storage, and retrieval (Hipp, 2025). Using SQLite, the system could accept a large collection of images from the user, pre-process and organize the images into a self-organized map according to the backend algorithm, then store the original images with its assigned node in a database for efficient retrieval.

The pre-processing method used in the preliminary CBIR system was limited primarily to identifying shapes and edges in images. To provide a more accurate and expanded extraction of features, I decided to explore feature extraction methods using deep neural networks. To keep the system operationally efficient, I used transfer learning to apply the weights derived from a pre-trained deep neural network model to produce vector embeddings that represent the extracted features from images. The use of transfer learning was facilitated by an image feature extraction pipeline provided by HuggingFace via the transformers Python package (Wolf et al., 2019). This pipeline provided a convenient way to use pre-trained models by bundling the pre-processing steps and neural network model into one package. Figure 3.5 describes the process by which transfer learning is used for

feature extraction. The pre-trained model from which the weights are borrowed is the vision transformer google/vit-base-patch16-224 developed by Dosovitskiy et al. (2020), which produced satisfactory results for a general-use system. This model was trained by processing images through a deep neural network, comparing the output with a target output, adjusting the weights to produce an output closer to the target, and repeating the process until an accurate output is achieved (Dosovitskiy et al., 2020).

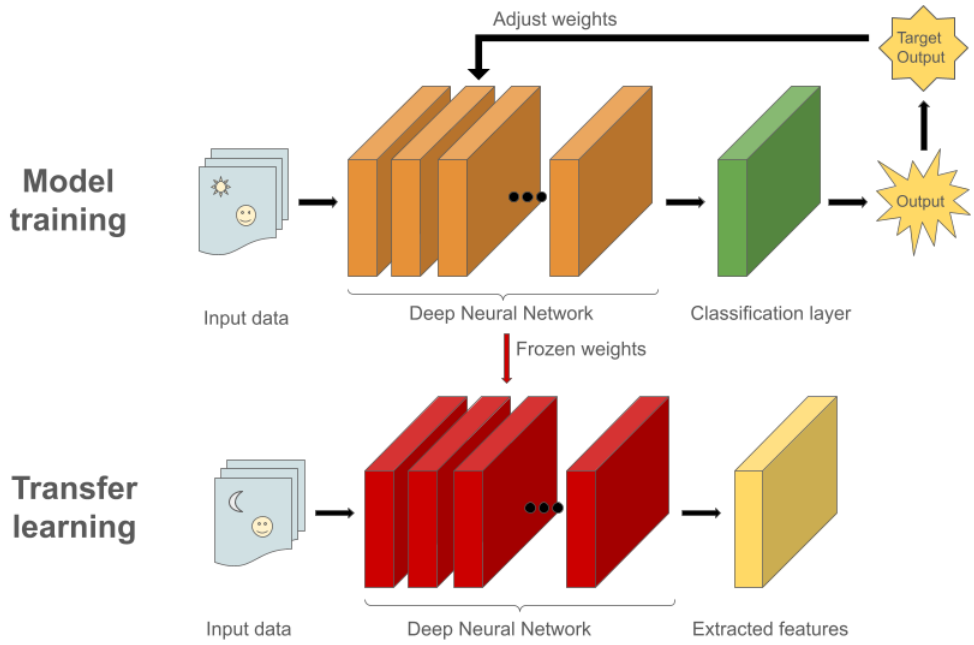

Figure 3.5: A deep neural network is trained to extract features from input data by comparing its output with a target output and adjusting the weights for each layer and neuron until an accurate output is achieved. These final weights can be reused in a transfer learning pipeline to extract features from a new input dataset.

The feature extraction pipeline produces a vector of 768 values, which could then be used as an input to an SOM algorithm in the same manner that the preliminary CBIR system used the 784-dimensional vectors of pixel values. By organizing images into a two-dimensional space based on measures of their features, an SOM algorithm reduces the

dimensionality of the feature space while clustering similar images together for more efficient retrieval. This concept is central to the creation of a computationally efficient design for DEBRIS.

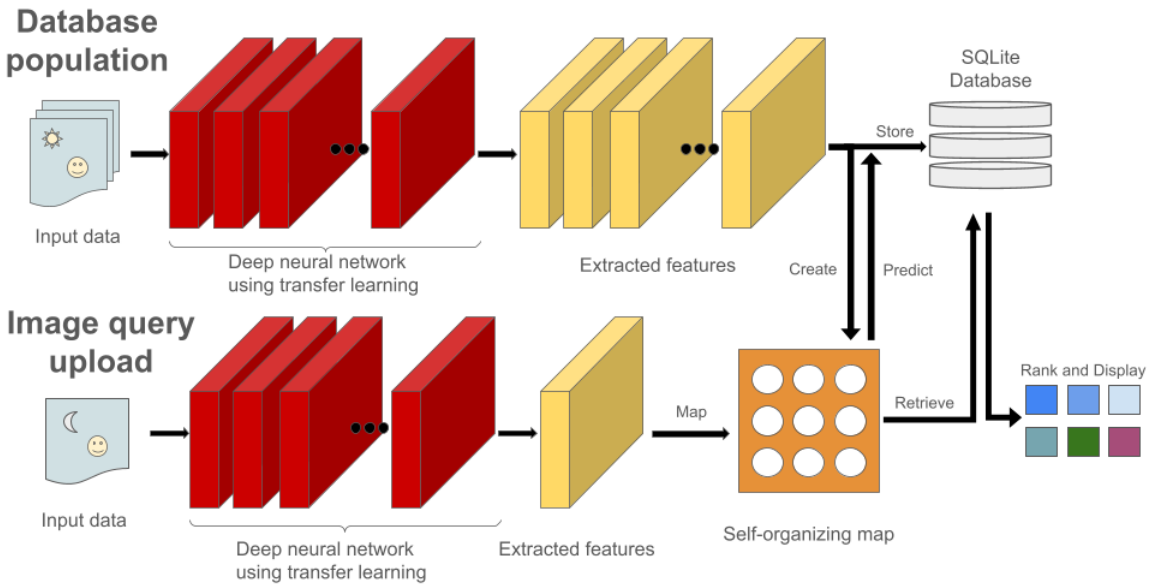

Figure 3.6: To populate a database for the DEBRIS application, the user uploads a collection of images to pass them through a feature extraction pipeline, which includes pre-processing steps and a pass through a pre-trained deep neural network model. The features extracted from each image are used to create a self-organizing map (SOM). Each image, its extracted features, and its associated node in the SOM are stored in an SQLite database. To browse the database, the user then uploads a single image, which is processed with the same feature extraction pipeline. The extracted features are used to map the image to the SOM and assign it to a node. Images in the database that are also associated to the node are retrieved, ranked, and displayed in order of similarity.

A final addition to the design of the DEBRIS web application is the inclusion of a ranked display of retrieved images. The preliminary CBIR system displayed a random selection of images retrieved from the node assigned to the image query. When DEBRIS retrieves images from the assigned node, it calculates the cosine similarity of the vector

embeddings between every image retrieved from the node and the image query. Up to seven images are then displayed in order of similarity ranking. Several more images are randomly selected and displayed from adjacent nodes to introduce variety and provide easier database exploration. The total number of images on display does not exceed ten. Figure 3.6 describes the operational flow of the DEBRIS application.

4

Results

When running the DEBRIS web application, the user is presented with an index page that displays a short description of DEBRIS and a drop-down menu to choose a database to browse. Two databases are provided as examples: Landscapes and Fashion. These databases were created from 10,000 images each, which were derived from datasets hosted on Kaggle and created by Ling et al. (2022) and Saxena (2022). The third option of the drop-down menu allows the user to upload their own image collection to create a custom database. Figure 4.1 shows the DEBRIS index page.

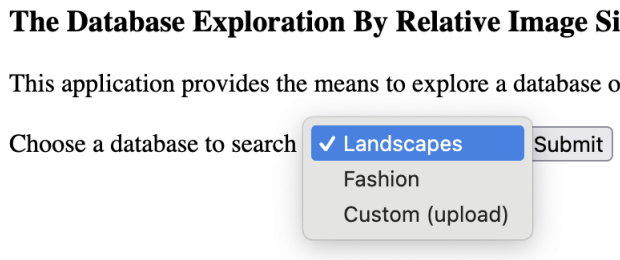

Figure 4.1: A screenshot of the index page.

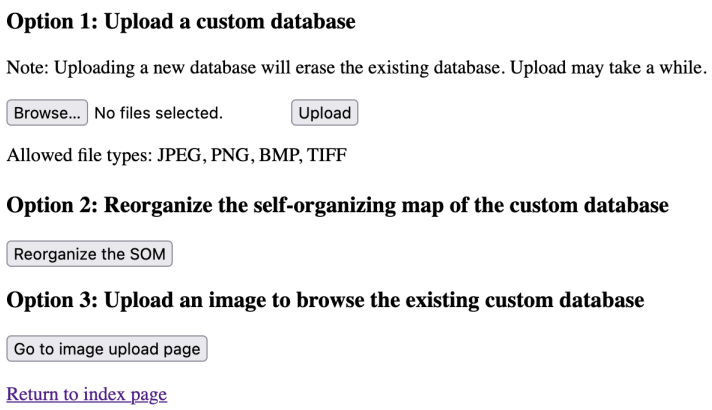

Figure 4.2: A screenshot of the custom database page. The user may choose to either upload a collection of images, reorganize the self-organizing map of the uploaded images, or visually browse the existing database by uploading an image.

When choosing to create a custom database, the user is directed to a page where they are given three options. The first option is to upload a collection of images to populate the database. If the database is already populated, the new uploads will overwrite it. The second option allows the user to reorganize the SOM if the database is already populated. The user may choose this option if the self-organizing map algorithm fails to run properly upon first upload, or if the user is not satisfied with the current organization of the map. The third option allows the user to browse the existing custom database. If the user has not previously uploaded any images to populate the database, no results will be returned. When a custom database is created, it is stored locally on the user's machine, so that if the user restarts the application, they can browse the previously created database without having to re-upload the entire image collection. Figure 4.2 shows a screenshot of this custom database page.

When uploading an image to browse by, the results are displayed as shown in Figure 4.3, which uses the example database named Fashion. These results show up to

seven images organized into the same node of the SOM, ranked by similarity score. Figure 4.3 demonstrates that the images retrieved take into account shape, color, and texture. The last three images, which are retrieved from adjacent nodes, are less similar, but they are still related to the image query, since they remain in the vicinity of the image query in the SOM space. From these retrieved images, the user may select the next image query. Figure 4.4 shows the results of choosing the ninth image in Figure 4.3 as the next image query. This figure demonstrates how a user may visually browse through the database using the DEBRIS application by continuing to select images for the next query. Another example of image search results using the example database named Landscapes is displayed in Figure 4.5.

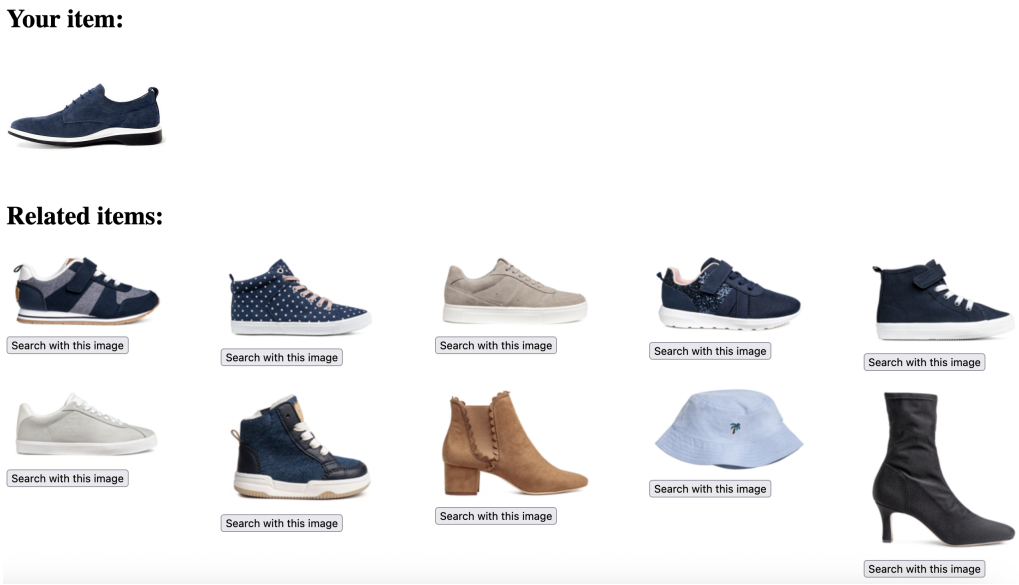

Figure 4.3: Result of uploading an image query to the DEBRIS application to browse the example database, Fashion. The first seven images were retrieved from the node assigned to the image query by the self-organizing map, and the last three images were retrieved from adjacent nodes.

Figure 4.4: Result of selecting an image query from images retrieved in Figure 4.3. This allows the user to browse the database further. The example database Fashion was used in this instance.

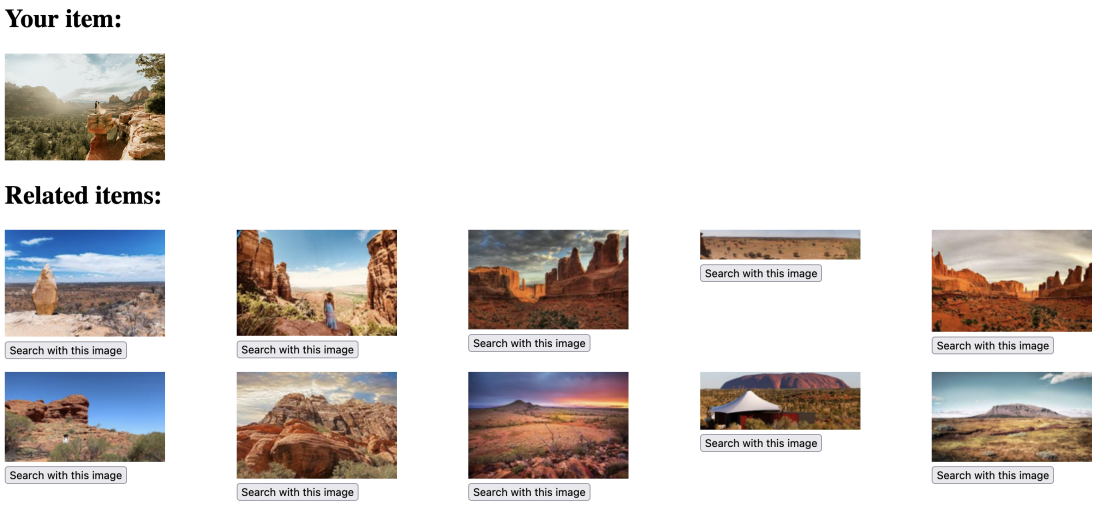

Figure 4.5: Result of uploading an image query to the DEBRIS application to browse the example database, Landscapes. The first seven images were retrieved from the node assigned to the image query by the self-organizing map, and the last three images were retrieved from adjacent nodes.

The SOMs created for each database are structured by a two-dimensional grid of 100 nodes, with 10 nodes along each dimension. Since the SOMs organize the uploaded images by similarity, they will not be evenly distributed across the nodes, especially if the image collection contains a large variety of sizes, textures, and colors. Figures 4.6 and 4.7 demonstrate how the SOM distributes the 10,000 uploaded images for the example databases, Fashion and Landscapes.

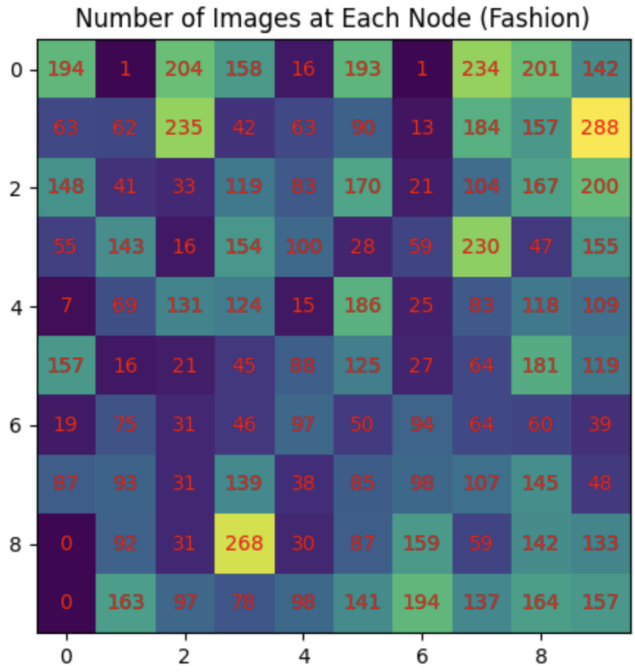

Figure 4.6: A self-organizing map's (SOM's) distribution of 10,000 uploaded images to create the Fashion database. The SOM is structured by a 10-by-10 grid of nodes. The Fashion database is provided as an example with the DEBRIS application.

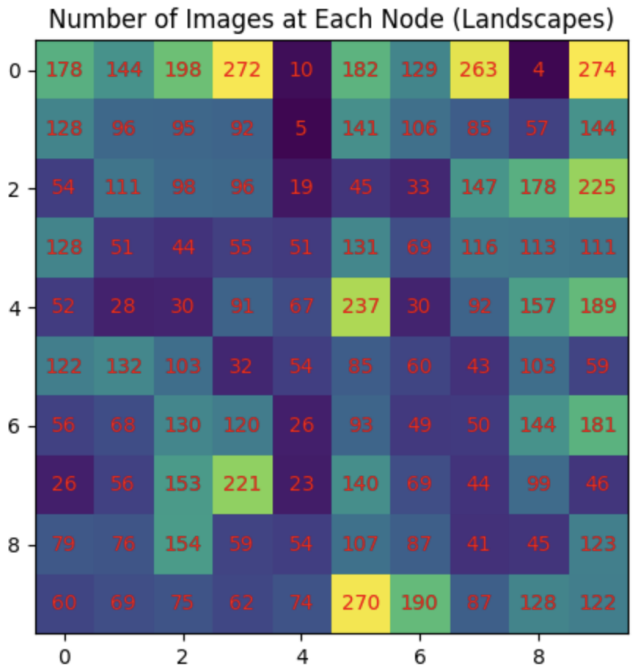

Figure 4.7: A self-organizing map's (SOM's) distribution of 10,000 uploaded images to create the Landscapes database. The SOM is structured by a 10-by-10 grid of nodes. The Landscapes database is provided as an example with the DEBRIS application.

Since the main purpose of the DEBRIS web application is to browse a database by image similarity, directed methods of browsing with varying degrees of randomness may be used to characterize the application's browsing ability. To illustrate this characterization, I have browsed the database by three methods: random selection of the ten images retrieved (random walk), random selection of the three images retrieved from adjacent nodes (directed random walk), and intentional selection of images that are the most subjectively dissimilar (user-directed walk). Each browsing session continued for 100 selections, so with 10 images retrieved for each selection, the maximum possible number of images retrieved is 1,000. While conducting these browsing experiments, the number of unique images encountered was tracked along with the path taken through each node to determine the breadth and

depth of the database exploration. The database used for this study was the Fashion example database.

To browse the Fashion database using DEBRIS by the random walk method, I began by uploading the image query used in Figure 4.3 with similar results. A random number generator function from the Python package NumPy was used to randomly produce a number from 1 to 10 (Harris et al., 2020). The result determined which image to select for the next round of retrieved images. In general, images 1 through 7 are retrieved from the same node, so a selection of one of these images keeps the user browsing through the same node. Images 8 through 10 are retrieved from adjacent nodes. A selection of one of these images allows the user to browse through a different node. If the current node does not contain at least 7 images, the remaining images, up to a total of 10, are retrieved from adjacent nodes, if enough images are available. As expected, the random walk directed a change of nodes approximately 33% of the time (34 node changes out of 100 selections). A total of 461 unique images were displayed out of the 10,000 total images displayed. Since the majority of images displayed are the most similar images to the query, the random walk chose similar images frequently, resulting in more than half of the total images retrieved being displayed multiple times. By selecting similar images frequently, the study also shows that exploration of the SOM, and by extension the database, is limited to a small portion of the data available for browsing. Figure 4.8 shows the result of the random walk, starting at node 20, returning to itself several times, and ending at node 3. The number of unique images displayed per node is illustrated in the figure by a color gradient, which is notably limited to a small portion of the map.

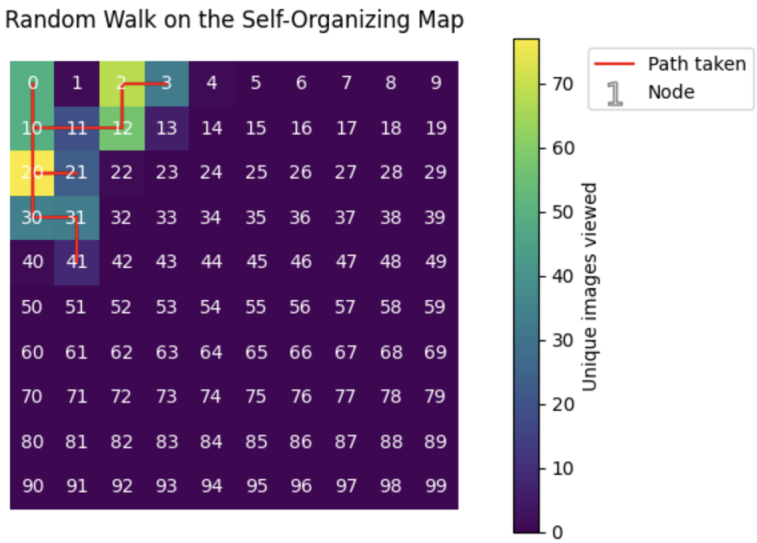

Figure 4.8: The results of navigating the Fashion database with the DEBRIS application by randomly selecting one of the retrieved images as the next query image. The random walk is plotted on the self-organizing map to show how the user might navigate through the data. Since most of the images available for selection are similar to the last query image, the random walk is limited in its reach.

To randomly browse the Fashion database using DEBRIS with the goal of retrieving as many unique images as possible, I performed an experiment similar to the previous random walk, except that the random numbers generated were limited to 8 through 10. This directed random walk ensured a change of nodes at every step, resulting in a path more akin to Brownian motion. For this experiment, 650 unique images were displayed out of a possible 1,000, which shows that the directed random walk provides a broader exploration of the data than the previous random walk. Since the images displayed from adjacent nodes are retrieved randomly, each selection is only loosely related to the image query, so the images similar to the selection are less likely to have been previously displayed. The display of images randomly selected from adjacent nodes provides the user with a means to broaden their exploration of the database away from their starting point, but given the

random nature of these retrieved images, a random selection may direct the user back toward the starting node. The path taken across the SOM using the directed random walk method demonstrates this broadened exploration of the database, where a choice between more similar and less similar results is made randomly (see Figure 4.9). The walk begins at node 20, touches nodes 42 and 0, then spends the rest of its steps alternating between nodes 12, 2, and 3, before finally ending on node 3.

To automatically browse the database by dissimilarity, a measure of dissimilarity among the displayed images would need to be provided. Since the application retrieves images randomly from adjacent nodes, no measure of dissimilarity is calculated in the algorithm, which means that it is up to the user's discretion to visually search the database for images that are less similar to the images presented at their current node. To demonstrate browsing across the database at the user's discretion, I performed a walk similar to the previous random walks, except that each step was intentionally taken to move away from the image query in terms of similarity. For example, in Figure 4.3, I would have chosen the ninth image, since it is the only image that is not footwear. This method of browsing resulted in the retrieval of 948 unique images out of a possible 1,000, which is evidence that a more extensive exploration of the database can be achieved with DEBRIS, as long as it is the user's intention to do so. Figure 4.10 shows the results of this last walk, starting with node 20.

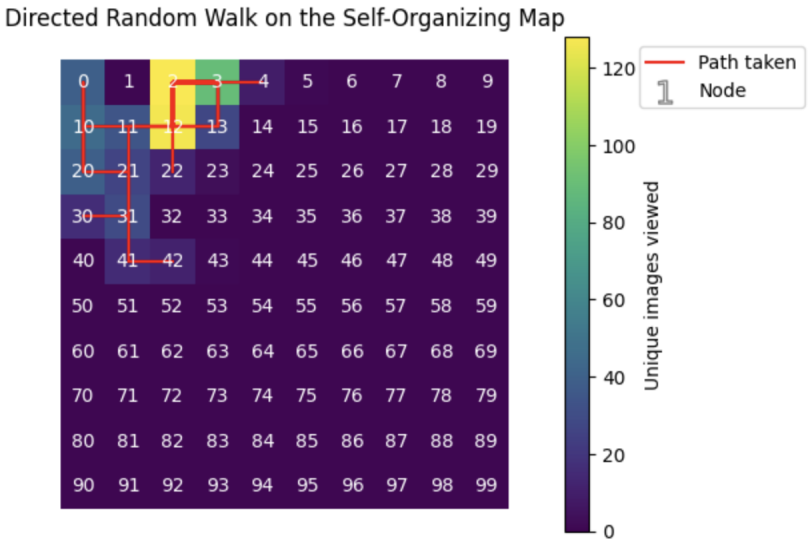

Figure 4.9: The results of navigating the Fashion database with the DEBRIS application by randomly selecting one of the retrieved images from adjacent nodes as the next query image. The random walk is plotted on the self-organizing map to show how the user might navigate through the data. Like the previous random walk, this walk is limited in its reach, but it extends further because it has a greater balance of choice between more similar and less similar images.

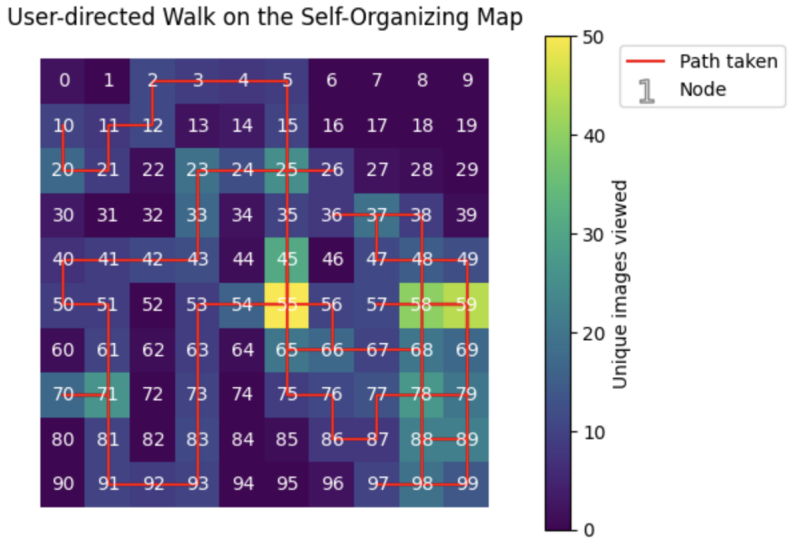

Figure 4.10: The results of navigating the Fashion database with the DEBRIS application by intentional selection of the most subjectively dissimilar image as the next query image. The walk is plotted on the self-organizing map to show how the user might navigate through the data. This demonstrates that the DEBRIS application may be used to thoroughly browse through a database if it is the user's intention.

5

Conclusion

This paper presents the DEBRIS web application, which is a CBIR system that provides a user with the ability to create an image database organized by similarity by uploading their own collection of images. The application then allows the user to visually search the custom database by uploading a single image as a query to retrieve similar images from the database. The user may continue browsing the database by selecting one of the images retrieved, which will then present a new collection of images similar to the selection. Since the images retrieved are displayed in order of similarity, the user may either choose to continue browsing within the space of similar images, or browse further from the image query by selecting less similar images. The results section describes the degree to which a user has the ability to browse a custom database using the DEBRIS application.

DEBRIS begins building custom databases by sending uploaded images to a convolutional neural network via a feature extraction pipeline. This pipeline uses transfer learning to automatically extract image features such as color, texture, and shape. The

extracted features for each image are then organized by a self-organizing map (SOM) into a two-dimensional grid of 10-by-10 nodes. The SOM maps the topology of the image collection in a two-dimensional space from the high-dimensional feature space. The resulting map contains clusters, where similar images are grouped together. These clusters are then stored in a database for convenient retrieval. When an image is uploaded as a query, its features are also extracted and mapped to the grid to retrieve images close to it in the SOM space. The structure of the SOM ensures that the images retrieved are visually similar to the image uploaded. The user may continue browsing by selecting one of the images displayed as the next image query.

The DEBRIS web application makes efficient use of machine learning algorithms with two key elements: the use of transfer learning regarding a deep neural network, and the use of a self-organizing map for data structuring. The use of transfer learning allows efficient feature extraction, since the weights used in the neural network are pre-trained. This eliminates the need to train the neural network before using it. The self-organizing map facilitates efficiency by organizing images in the database by similarity. This eliminates the need to analyze all of the data in order to retrieve similar images.

Future work on this web application could include a study on the effects of using different vision transformers. Additionally, development of the application could include a refined user experience and added robustness for the efficient creation of larger databases. Lastly, more immediate development could include the ability for a user to save multiple databases and the ability to add images to an existing database. The DEBRIS application is open source and free to use. It can be found at https://github.com/san-ford/DEBRIS.

Bibliography

Django Software Foundation. (2025). *Django* (Version 5.2.4). https://www.djangoproject.com/

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.* https://doi.org/10.48550/arxiv.2010.11929

Gaata, M. T., & Fouad Hantoosh, F. (2018). Encrypted Image Retrieval System based on Features Analysis. *Al-Mustansiriyah Journal of Science, 28*(3), 166–173. https://doi.org/10.23851/mjs.v28i3.77

Gaffney, K. P., Prammer, M., Brasfield, L., Hipp, D. R., Kennedy, D., & Patel, J. M. (2022). SQLite: past, present, and future. *Proceedings of the VLDB Endowment, 15*(12), 3535–3547. https://doi.org/10.14778/3554821.3554842

Gaillard, M., Egyed-Zsigmond, E., & Granitzer, M. (2018). CNN features for Reverse Image Search. *Document Numérique, 21*(1–2), 63–90. https://doi.org/10.3166/RIA.21.1-31

Hai, N. M., Lang, T. V., & The Van, T. (2023). Improving the Efficiency of Semantic Image Retrieval Using a Combined Graph and SOM Model. *IEEE Access, 11*, 140646–140659. https://doi.org/10.1109/ACCESS.2023.3333678

Hameed, I. M., Abdulhussain, S. H., & Mahmmod, B. M. (2021). Content-based image retrieval: A review of recent trends. *Cogent Engineering, 8*(1). https://doi.org/10.1080/23311916.2021.1927469

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). *Array programming with NumPy. Nature (London), 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition.* https://doi.org/10.48550/arxiv.1512.03385

Heesch, D. (2008). A survey of browsing models for content based image retrieval. *Multimedia Tools and Applications, 40*(2), 261–284. https://doi.org/10.1007/s11042-008-0207-2

Hipp, D. R. (2025). *SQLite* (Version 3.50.4). https://sqlite.org/

Katasani, S. R., Rachepalli, T., Kamat, N., & Jadhav, M. (2019). Similar Fashion Finder using Reverse Image Search. *International Journal of Computer Sciences and Engineering, 7*(5), 1190–1195. https://doi.org/10.26438/ijcse/v7i5.11901195

Kiran, A., Qureshi, S. A., Khan, A., Mahmood, S., Idrees, M., Saeed, A., Assam, M., Refaai, M. R. A., & Mohamed, A. (2022). Reverse Image Search Using Deep Unsupervised Generative Learning and Deep Convolutional Neural Network. *Applied Sciences, 12*(10), 4943. https://doi.org/10.3390/app12104943

Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological Cybernetics, 43*(1), 59–69. https://doi.org/10.1007/bf00337288

Laaksonen, J., Koskela, M., Oja, E., Smeulders, A. W. M., & Huijsmans, D. P. (1999). Content-Based Image Retrieval Using Self-Organizing Maps. In *Visual Information and Information Systems* (pp. 541–549). Springer Berlin Heidelberg. https://doi.org/10.1007/3-540-48762-X_67

Laaksonen, J. T., Markus Koskela, J., & Oja, E. (2004). Class distributions on SOM surfaces for feature extraction and object retrieval. *Neural Networks, 17(*8), 1121–1133. https://doi.org/10.1016/j.neunet.2004.07.007

Lin, C., Tsai, C., & Roan, J. (2008). Personal photo browsing and retrieval by clustering techniques: Effectiveness and efficiency evaluation. *Online Information Review, 32*(6), 759–772. https://doi.org/10.1108/14684520810923926

Ling C. G., ElizabethHMGroup, FridaRim, inversion, Ferrando, J., Maggie, neuraloverflow, & xlsrln. (2022). *H&M Personalized Fashion Recommendations* [Data set]. Kaggle. https://kaggle.com/competitions/h-and-m-personalized-fashion-recommendations

Meng, X., An, Y., He, J., Zhuo, Z., Wu, H., & Gao, X. (2016). Similar image retrieval only using one image. *Optik (Stuttgart), 127*(1), 141–144. https://doi.org/10.1016/j.ijleo.2015.10.041

Pallets (2025). *Flask* (Version 3.1.1). https://palletsprojects.com/projects/flask/

Rafferty, P. (2019). Disrupting the Metanarrative: A Little History of Image Indexing and Retrieval. *Knowledge Organization, 46*(1), 4–14. https://doi.org/10.5771/0943-7444-2019-1-4

Sampathila, N., Pavithra, & Martis, R. J. (2022). Computational approach for content-based image retrieval of K-similar images from brain MR image database. *Expert Systems, 39*(7). https://doi.org/10.1111/exsy.12652

Saxena, U. (2022). *Landscape classification dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/utkarshsaxenadn/landscape-recognition-image-dataset-12k-images

Seo, E., Kim, S., Lee, Y., Han, S.-I., Kim, H.-S., Rey, S.-C., & Song, H. (2023). Similar Image Retrieval using Autoencoder. I. Automatic Morphology Classification of Galaxies. *Publications of the Astronomical Society of the Pacific, 135*(1050), 84101. https://doi.org/10.1088/1538-3873/ace851

Singh, D., Mathew, J., Agarwal, M., & Govind, M. (2023). DLIRIR: Deep learning based improved reverse image retrieval. *Engineering Applications of Artificial Intelligence, 126*, Article 106833. https://doi.org/10.1016/j.engappai.2023.106833

Smith, R. P. (2021). *sklearn-som* (Version 1.1.0). https://github.com/rileypsmith/sklearn-som

Wang, C., Reese, J. P., Zhang, H., Tao, J., Gu, Y., Ma, J., & Nemiroff, R. J. (2015). Similarity-based visualization of large image collections. *Information Visualization, 14*(3), 183–203. https://doi.org/10.1177/1473871613498519

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T. L., Gugger, S., ... Rush, A. M. (2019). HuggingFace's Transformers: State-of-the-art Natural Language Processing. https://doi.org/10.48550/arxiv.1910.03771

Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. https://doi.org/10.48550/arxiv.1708.07747